

Title	AXI Internal SRAM Controller
Project	AXI
Description	AXI BUS 용 Internal SRAM Controller 설계
Author	holelee
Date	19/Apr/2005

(*) AXI Internal SRAM Controller

AXI는 기본적으로 SRMD(Single Request Multiple Data)를 사용하고, Read Channel과 Write Channel이 완전히 구분되는 구조를 가지고 있으므로 Internal SRAM Controller 마저도 그렇게 간단하게 설계할 수 없다. ARM에서 제공하는 Internal SRAM Controller를 살펴보고 그에 맞추어 어떻게 설계할 수 있는지 생각해 본다.

(*) ARM AXI Internal Memory Interface

ARM에서 제공하는 AXI Internal Memory Interface의 주요 특징은 다음과 같다.

- Single Port SRAM or ROM
- Round-robin arbitration between read and write transaction
- For write transactions, the first data transfer can take place 2 cycles after the address is accepted. Subsequent data transfers can complete in consecutive cycles.
- For read transactions:
 - With one wait state
 - The first data transfer can take place 2 cycles after the address is accepted. Subsequent data transfers can complete in consecutive cycles.
 - With zero wait state
 - The first data transfer can take place 3 cycles after the address is accepted. Subsequent data transfers take 2 cycles
- It supports
 - all AMBA AXI channels except the low power channel
 - all AXI burst types
 - aligned and unaligned transfer types
- You can configure the following parameters
 - data width of 64 bits or 32 bits
 - ID width
 - wait states, single or none, for SRAM and ROM reads
 - whether the interface is to be used with SRAM or ROM
 - memory address width

재미 있는 것은 zero wait state memory(asynchronous memory)를 지원한다는 점이다. 하지만 더이상 asynchronous internal SRAM을 사용하는 경우는 거의 없으므로 우리는

synchronous SRAM만을 지원하도록 한다. ROM의 경우는 아마도 write channel에 request를 보내면 error response를 보낼 것으로 보이는데, 우리는 그렇게 하지 않고 ROM의 경우 read channel만 남기고, write request의 경우는 AXI BUS에서 default slave로 routing하는 식으로 해결하려고 한다.

(*) 구조

전체적인 구조는 ARM에서 제공하는 것과 같이 다음 그림 처럼 구성된다..

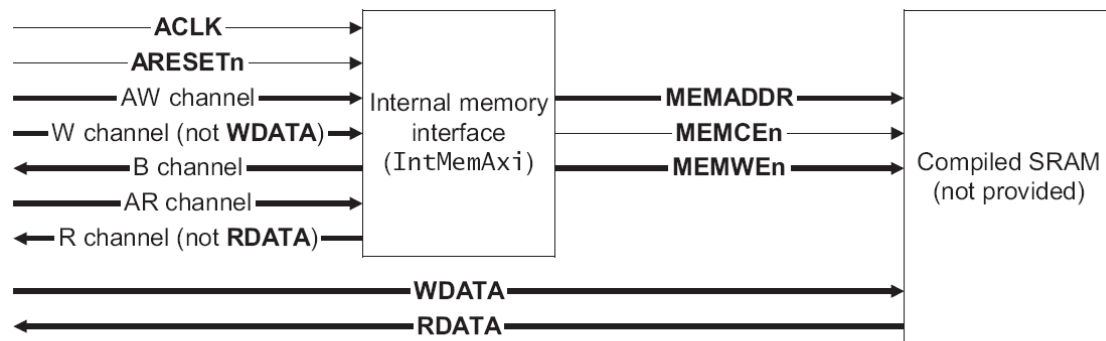


Figure 1 Internal memory interface connected to compiled SRAM

WDATA와 RDATA는 직접 AXI interface에서 SRAM쪽으로 전달된다. 그리고 나머지 signal은 Internal SRAM Controller쪽에서 받아서 처리한 후에 SRAM쪽으로는 MEMADDR과 MEMCEn, MEMWEn(Byte Width에 맞추어)를 출력해 주게 된다.

WDATA와 RDATA가 직접 전달되는 것이 독특한 데, RDATA의 경우 주의가 필요하다. RDATA가 SRAM에서 Interface로 직접 전달되는 구조를 가지려면, MEMADDR이 register로 구성되면 안되고 반드시 RREADY와 combinational logic으로 이루어져야 한다. Synchronous SRAM은 address를 주면 다음 clock에서 data를 채는 형태를 가지므로, RREADY가 1이면 MEMADDR이 다음 읽을 address를 출력해야 하고, RREADY가 0이면 현재 address를 유지해야 하기 때문이다.

AW Channel, AR Channel의 signal 중에 다음과 같은 것은 받지 않는다.

- AWLOCK/ARLOCK : 즉 exclusive access는 지원하지 않음.
- AWCACHE/ARCACHE : 항상 Non-cache
- AWPROT/ARPROT : Protection 없음

(*) Main State Machine

기본적으로 전체적인 State는 다음과 같이 네 가지가 있다.

- S_IDLE : idle state

AR channel 혹은 AW channel에 READY를 high로 하고 있다. READY는 VALID 입력의 combinational output이 되면 안된다. Arbitration state에 맞추어 한 쪽 channel에 READY를

출력하고 있다가, 한 쪽 channel이 먼저 VALID하게 되면 그 다음 cycle에 해당 channel을 READY로 하면 된다. VALID와 READY가 동시에 high이면 address및 기타 정보를 latch하고 READ/WRITE에 맞추어 S_READ/S_WRITE state로 옮겨 간다.

- S_READ : read state

READ state는 Address generation logic의 출력이 last가 될 때 까지 기다린다. READ State에서는 RVALID가 항상 1이고 MEMCEn이 항상 0이 된다. RRESP는 state와 관계 없이 항상 OKAY를 assign한다.

- S_WRITE : write state

WRITE state는 Write Data Channel에서 last가 될 때 까지 기다린다. WRITE State에서는 WREADY가 항상 1이고, MEMCEn은 WVALID가 1일 때 0, MEMWEn은 WVALID와 WSTRB를 참조하여 생성하게 된다.

- S_WRESP : write response state

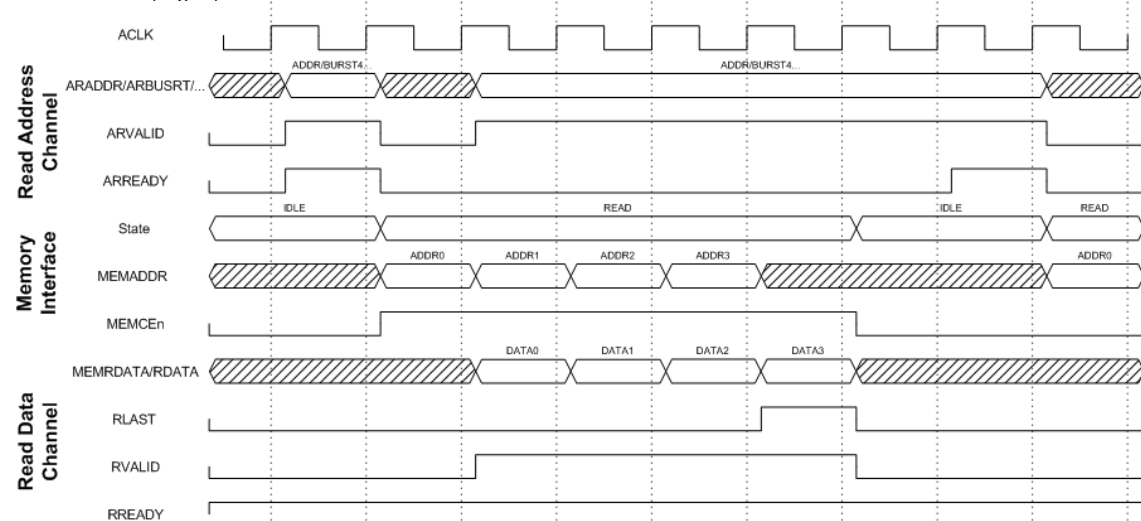
WRESP(Write Response) state는 S_WRITE 다음에 지나는 state로 BRESP에 OKAY 신호를 주고 BVALID를 항상 1로 한다.

(*) Address Generation Logic

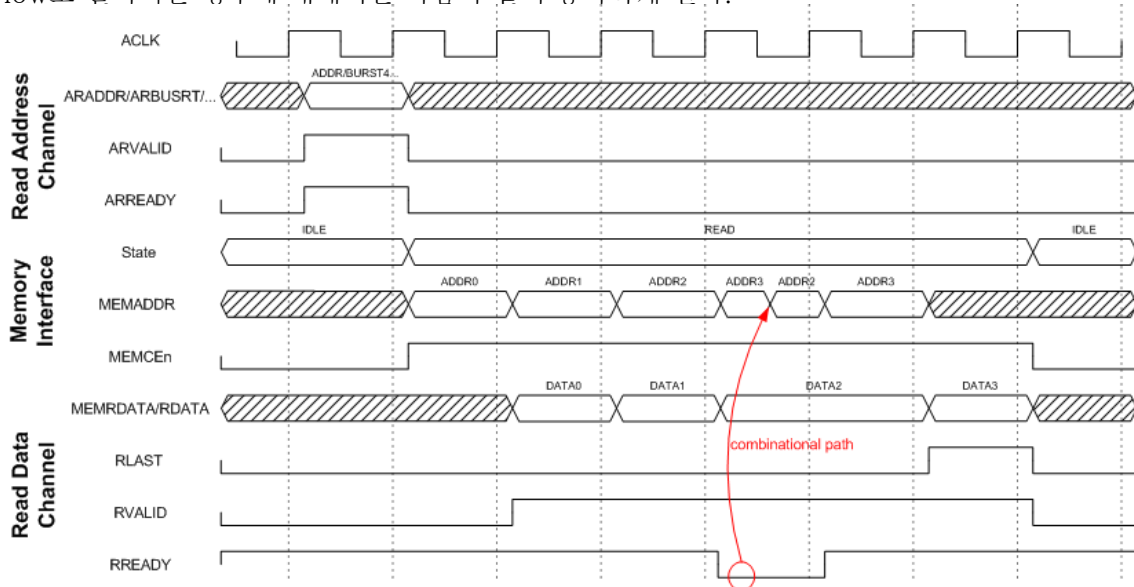
Address Generation Logic은 S_READ/S_WRITE state에서만 동작한다. 출력은 MEMADDR과 last data signal(READ를 위해)이며 READ의 경우 RVALID/RREADY, WRITE의 경우 WVALID/WREADY가 동시에 high일 때 다음 address를 생성하고 len를 감소한다. len가 0이면 last signal을 생성한다. address 생성은 Burst length, Burst Size, Burst Type 정보를 이용하여 하게 된다. AXI는 기본적으로 4KB boundary를 넘어서는 burst를 지원하지 않는데, Address Generation Logic도 그렇게 코딩하기로 한다.

(*) Internal SRAM Controller의 동작

- READ의 동작



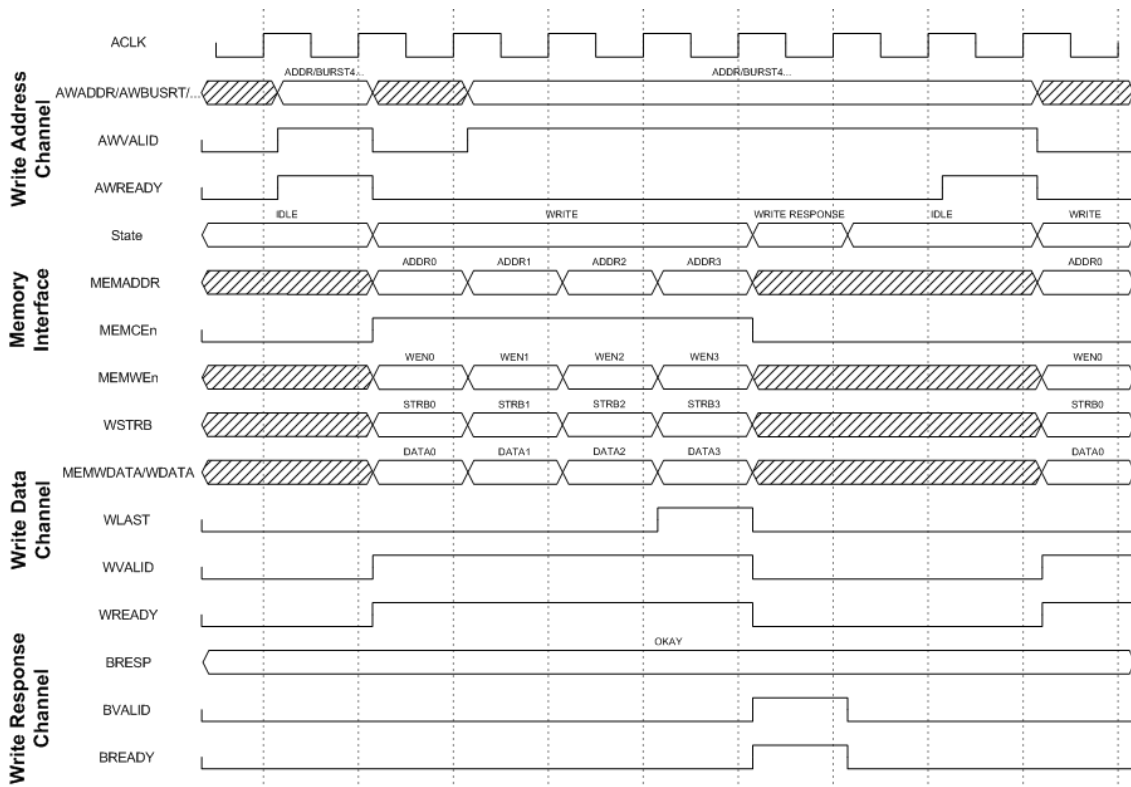
기본적인 Read의 동작은 위와 같다. Read Address Channel의 request가 accept되고 1 cycle의 delay를 가진 다음 매 cycle data 전송이 이루어진다. 만약 MASTER의 RREADY가 중간에 low로 떨어지는 경우에 대해서는 다음과 같이 동작하게 된다.



RREADY가 low인 high인지가 MEMADDR의 출력과 combinational path를 이루고 있다. 즉 SRAM Controller의 FSM은 Mealy machine이다. 이런 구조는 Memory에서 읽은 MEMRDATA와 RDATA가 그냥 wire로 연결되는 경우 피할 수 없다. SRAM Controller에서 Memory의 출력인 MEMRDATA를 latch하여 AXI Master로 전달하는 구조로 되어 있다면 위와 같은 combinational path를 피할 수 있지만 그런 경우에는 latency가 2로 늘어나게 되는 단점이 생긴다. 어쨌든 MEMADDR의 값을 clock의 rising edge에서 SRAM에서 latch하게 되므로 timing상 큰 문제를 일으키지 않으리라 생각된다.

- WRITE 동작

Write 동작의 기본적인 형태는 다음과 같다. Read와 달리 Write는 Request를 accept하고 바로 다음 cycle부터 동작이 시작된다. Request를 accept할 때(AWREADY가 high) WVALID의 값을 조사하지 않는다.



MEMWEn 신호는WSTRB 신호를 이용하여 combinational한 출력을 하게 된다.

- Arbitration

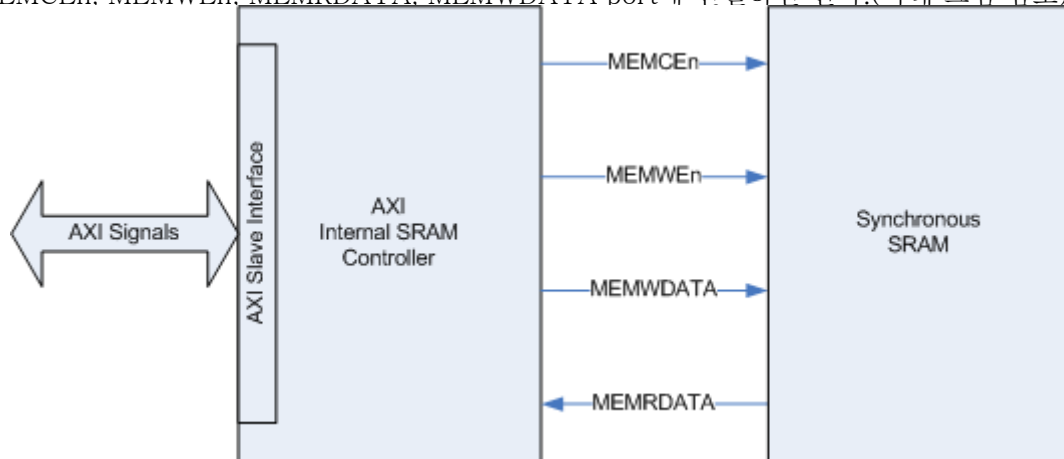
READ와 WRITE를 arbitration하는 것은 간단한 방법을 사용한다. 우선 ReadOrWrite라는 flag register가 있어서 그 값을 toggle하면서 round-robin arbitration을 구현한다. IDLE state일 때 ReadOrWrite의 값이 0이면 ARREADY가 assert되고 1이라면 AWREADY가 assert된다. AWVALID/ARVALID를 직접 combinational하게 이용하지 않은 이유는 그것이 AXI의 권장사항(?)이기 때문이다. 또한 일단 Request를 accept하게 되면 ReadOrWrite가 toggle하여 다음 request는 반대쪽 request(read를 accept하면 write, write를 accept하면 read)에 우선 순위를 가지도록 한다. 만약 IDLE state에서 ReadOrWrite가 0인데 ARVALID는 low이고 AWVALID가 high라면 다음 cycle에서 ReadOrWrite가 toggle된다. 반대의 경우도 마찬가지이다. 위의 Read/Write 예제에서 state가 IDLE이 되었는데도 바로 request를 accept하지 못하고 1 cycle을 기다리는 이유가 바로 ReadOrWrite의 toggle때문이다.

사실 IDLE일 때 뿐 아니라 Write Response state이거나, 맨 마지막 read data가 전달 되는 부분에서 이러한 toggle을 수행하면 1 cycle을 줄일 가능성이 있으나 지금 구현에서는 그렇게 하지 않았다.

(*) SRAM Controller의 사용법

- 중요한 parameter
- port 연결

기본적으로 AXI signal을 모두 연결하고, 합성된 Synchronous SRAM을 MEMADDR, MEMCEn, MEMWEn, MEMRDATA, MEMWDATA port에 연결하면 된다.(아래 그림 참조)



(*) SRAM Controller의 제한 점

위에서 언급한 Arbitration latency가 1 cycle을 제거할 필요가 있고, Read back-to-back transfer가 지원되지 않는다는 점을 고쳐야 한다.